Gate 2: Bezhlavé spouštění a samoopravný běh

Architektura, bezpečnostní mechanismy a empirické vyhodnocení GrassAgent

Bc. Mykhailo Radchenko

Projekt GrassAgent

Říjen 2026

Obsah

1. Vymezení rozsahu
2. Integrita benchmarku a audit úniku
3. Architektura a izolace
4. Výsledky a analýza
5. Diskuze a otázky

Důvody pro nezařazení add-onů a tutoriálů

Proč nezařazujeme Add-ony (`g.extension`)

- **Přehlcení RAG dohledávání**: Stovky neoficiálních modulů degradují přesnost vektorového vyhledávání v korpusu.
- **Nároky na prostředí**: Vyžadují dynamickou instalaci a kompilaci, což narušuje deterministický offline-first běh.
- **Nestabilita rozhraní**: Nejednotná kvalita kódu a chybějící garance podpory `grass.tools`.

Proč nezařazujeme externí Tutoriály

- **Zastaralé rozhraní**: Většina tutoriálů používá legacy `grass.script` nebo CLI místo moderního `grass.tools`.
- **Nízká použitelnost**: Vhodných návodů je naprosté minimum; zbytek trpí jazykovou nejednotností (mimo angličtinu).
- **Chybějící ground truth**: Návody neobsahují standardizované metriky pro automatické vyhodnocení.

Výsledek pro spolehlivý benchmark

Soustředění na 543 oficiálních nativních nástrojů v `grass.tools` a 40 přesně definovaných úloh nad certifikovaným datasetem North Carolina.

Audit úniku dat: jak funguje `audit_leakage.py`

Model hrozeb

Riziko: únik receptu z promptu do korpusu zlepšuje zdánlivě kvalitu vyhledávání. Cílová řešení, očekávané výstupy a vnitřní stav nesmí být v promptu.

Mechanismus

- Korpus: 543 popisů nástrojů z `data/grass_knowledge.db`.
- Jaccardova podobnost obsahových slov + 3-/4-gramy.
- Test specifičnosti: $\geq$3 obsahová slova + rarita.
- Prahová hodnota 0,35; překročení znamená ruční kontrolu.

Naměřený překryv (40 případů)

- Průměrný Jaccard: 0,163.
- Maximum: 0,385 (`tc_04` vs. `d.to.rast`).
- Nad prahem: 2/40; oba případy označeny CLEAN.
- Označení `POTENTIAL_LEAK`: 1/40 (`tc_35`).

Síťová a promptová izolace: Testy zachytávají `socket.connect` a `socket.create_connection`; jakýkoli odchozí pokus selže. Zero-Chat vrací pouze spustitelný Python.

Prompty vyžadující ruční kontrolu

Audit porovnává prompt s korpusem, nikoli s očekávaným nástrojem. První dva případy překračují práh 0,35; třetí je jediné označení `POTENTIAL_LEAK`.

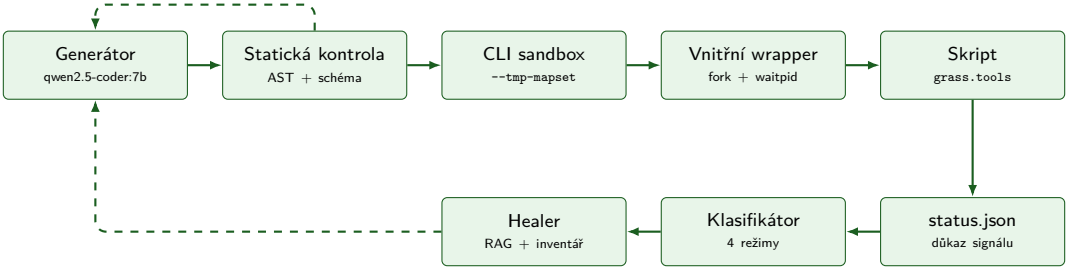
ID	Prompt (originál)	Nejbližší nástroj	Jaccard	Verdikt
`tc_04`	*"Display the elevation raster map on the active monitor."*	`d.to.rast`	0,385	CLEAN; nad prahem
`tc_02`	*"Calculate slope and aspect maps from the elevation raster."*	`r.slope.aspect`	0,357	CLEAN; nad prahem
`tc_35`	*"Interpolate a continuous elevation surface raster from spot elevation vector points using regularized spline with tension."*	`r.resamp.rst`	0,174	`POTENTIAL_LEAK`; označeno

Interpretace

Shoda nad prahem není důkaz kontaminace. U `tc_04` a `tc_02` jde o věcné popisy algoritmů, proto jsou CLEAN. U `tc_35` se opakuje specifická fráze z popisu `r.resamp.rst`, ale očekávaný nástroj je `v.surf.rst`; případ je třeba ručně posoudit.

Prompty jsou v původním anglickém znění; překlad by změnil podobnostní metriku.

Architektura Gate 2: přehled



Jak číst diagram

Plná šipka je tok jednoho pokusu; čárkované šipky jsou opravné cykly (horní při zamítnutí statickou kontrolou, spodní po selhání při běhu v sandboxu).

Uzel 1: Generátor

Role

- Lokální model `qwen2.5-coder:7b` přes Ollama.
- Generuje první pokus i opravy jako Python skript.
- Nemá přístup k internetu ani ke stavu verifieru.

Vstupy a výstupy

- Vstup: zadání + dohledané dokumenty + inventář map.
- Výstup: pouze spustitelný Python (Zero-Chat).
- Cílové řešení a očekávané výstupy nejsou v promptu.

Blok inventáře v promptu

```
### Available Dataset Maps
These are the ONLY maps that exist in this dataset.
```

Proč je uzel rizikový

Generování je stochastické. Statická validita 90 % neznamená spustitelnost; první pokus uspěje jen u 27,5 % případů.

Uzel 2: Statická kontrola

Role

Kontrola před spuštěním: parsování AST, validace schématu a vynucení rozhraní `grass.tools`. Odmítne neparsovatelný kód, legacy `grass.script`, shell i řetězená volání.

Co se kontroluje

- Syntaktická správnost Pythonu.
- Pouze `from grass.tools import Tools`.
- Žádné `run_command`, `subprocess` ani `os.system`.
- Zero-Chat: žádné Markdown bloky ani próza.

Co statická kontrola nezachytí

- Dynamické chyby až při běhu: chybějící mapa, parametry, region.
- Statická validita 90 % vs. Pass@1 27,5 %.

Důkaz

Integrační smoke testy přijímají správný skript a odmítají neparsovatelný kód, legacy rozhraní i řetězená volání.

Uzel 3: CLI sandbox

Role

Primární spuštění: `grass --tmp-mapset <dataset> --exec python3`. Každý pokus dostane dočasný mapset a vlastní scratch adresář.

Izolace běhu

- Dočasný mapset `--tmp-mapset`.
- Neprivilegovaný uživatel `grader`.
- Časový limit 120 s; `killpg` ukončí celou skupinu.
- Úklid `tmp_*` a `TMPDIR`.

Trvalá data

- Zákaz zápisu `chmod -w` na `PERMANENT`.
- SHA-256 hash před a po běhu.
- Nulový počet zanechaných scratch složek.

`--tmp-mapset` není bezpečnostní hranice

Skript může přepnout mapset a zapisovat do něj. Skutečnou izolaci tvoří práva na souborovém systému a kontrola hashe.

Uzel 4: Vnitřní wrapper

Role

`inner_wrapper.py` spouští potomka přes `fork` a `waitpid` a zapisuje `status.json`. Cílem je rozlišit pád na úrovni C od explicitního `sys.exit(245)`.

Co wrapper zachytí

- Exit kód a jméno signálu (`SIGSEGV`, `SIGABRT`).
- Výstup na stdout a stderr.
- Konec potomka nezávisle na navazujících procesech.

Proč nestačí exit kód

- Kód 245 může znamenat pád i vědomé ukončení.
- Bez signálu nelze pád spolehlivě klasifikovat.
- `status.json` je jediný zdroj pravdy.

Důkaz

Test se záměrným segfaultem zapíše `outcome: crashed` a `signal_name: SIGSEGV`; nevychází přitom z kódu 245.

Uzel 5: Generovaný skript

Role

Spustitelný Python, který volá pouze moderní `grass.tools`. Skript se ukládá jako artefakt a je znovu použitelný pro audit a reprodukci.

Povoleno

- `from grass.tools import Tools`.
- Volání typu `tools.r_slope_aspect(...)`.
- Čistý Python bez Markdownu a prózy.

Zakázáno

- `grass.script.run_command`.
- `subprocess`, `os.system`, shell.
- Řetězená volání a legacy rozhraní.

Důkaz

Automatický audit skenuje uložené artefakty a selže při jakémkoli zakázaném volání.

Uzel 6: `status.json`

Role

Strojově čitelný důkaz o průběhu pokusu: výsledek, exit kód, signál a zachycený výstup. Klasifikátor z něj určuje režim selhání.

Outcome	Význam	Režim
`executed_ok`	skript doběhl s kódem 0	úspěch
`runtime_failed`	běh skončil chybou	dle stderr
`crashed`	signál na úrovni C	`general_exception`
`timeout`	překročen limit 120 s	`general_exception`

Proč samostatný soubor

Exit kód sám nerozliší pád a vědomé ukončení. `status.json` zachová signál i kontext pro reprodukovatelnou klasifikaci.

Uzel 7: Klasifikátor

Role

Přiřadí každému selhanému pokusu právě jeden ze čtyř režimů. Pořadí je pevné, klasifikace je deterministická a idempotentní; žádné selhání nesmí zůstat nezařazené.

Precedence: `unknown_method` → `missing_input` → `schema_misuse` → `general_exception`.

Režim	Typický signál	Počet
`missing_input`	mapa nebo soubor neexistuje	31
`schema_misuse`	chybějící nebo neplatný parametr	17
`unknown_method`	neznámý nástroj nebo metoda	0
`general_exception`	pád, timeout, neošetřená chyba	35

Důkaz

112 testů klasifikátoru pokrývá precedence, lokalizované chyby a idempotenci; všech 83 selhaných pokusů je klasifikováno.

Uzel 8: Healer (opravný cyklus)

Role a mechanismus

- Max. 2 opakování; každé selhání spotřebuje jeden pokus.
- RAG nad 543 nástroji + cílený dotaz na selhávající nástroj.
- Opravný prompt nese 4 prvky: chybu, dokumentaci, kód a inventář.
- Zastaví se při úspěchu nebo vyčerpání rozpočtu.

Ukotvení v inventáři map

- `inventory.py` čte `PERMANENT/cellhd` a `vector` přímo.
- Bez GRASS relace; max. 80 map; záložní `g.list`.
- Inventář jde do 1. pokusu i do všech oprav.
- Nečitelné mapy se vynechají, nikdy se nevymýšlejí.

Naměřený dopad a limit

`missing_input`: 56 → 31 (−45 %); healing gain +5,0 p. b. Opakovaná stejná selhání však dominují (49 z 54); další zlepšení vyžaduje ukotvení parametrů, ne jen více opakování.

Hlavní výsledky: statická vs. dynamická validita

Měření	Případy	Úspěšnost
Gate 1 statická validita (baseline)	36/40	90,0 %
Gate 2 Pass@1 (zero-shot)	11/40	27,5 %
Gate 2 Pass@Healed (max. 2 opakování)	13/40	32,5 %
Nárůst díky opravám (healing gain)	**+2 případy**	**+5,0 p. b.**

Klíčový poznatek

Statická analýza AST a validace schématu jsou slabými ukazateli spustitelnosti v GIS: statická validita dosahuje 90,0 %, ale pouze 27,5 % skriptů reálně proběhne na první pokus.

Přínos self-healingu

+5,0 procentního bodu (+18,2 % relativně): 2 dodatečné případy opraveno v rámci limitu 2 opakování.

Pass@Healed měří spustitelnost, nikoli geografickou správnost.

Odhalení skrytých chyb (Efekt inventáře map)

Režim selhání (s inventářem)	Pokusy
`general_exception`	35
`missing_input`	31
`schema_misuse`	17
Celkem neúspěšných pokusů	**83**

*Součet neúspěšných pokusů napříč všemi opakováními (11/40 případů prošlo hned na 1. pokus).

Srovnání 2 běhů (bez vs. s inventářem v 1. promptu)

- **Vyřešeno (halucinace vrstev):** `missing_input` klesl z 56 na 31 (−45 %) díky injektování inventáře map hned do 1. promptu.
- **Odhaleno (hlubší chyby):** `general_exception` vzrostl z 10 na 35. Skripty došly dále a odhalily chyby v parametrech:
  - `v.to.rast` bez parametru `column=`
  - `d.rast` bez monitoru `d.mon`
  - Chybějící nastavení regionu (`g.region`)

Interpretace výsledků (Error Unmasking)

Zavedení inventáře map do 1. promptu vyřešilo halucinace vrstev, což odmaskovalo hlubší vrstvu parametrů nástrojů. Jde o diagnostický pokrok, který předchází celkovému nárůstu úspěšnosti.

Přechody mezi režimy selhání

Přechod	Význam přechodu	Počet
`general_exception` → `general_exception`	Přetrvává stejná chyba	21
`missing_input` → `missing_input`	Přetrvává chybějící vstup	20
`schema_misuse` → `schema_misuse`	Přetrvává chyba schématu	8
`schema_misuse` → `general_exception`	Odhalena hlubší chyba	3
`general_exception` → `success`	Úspěšně opraveno	1
`schema_misuse` → `success`	Úspěšně opraveno	1

Vyhodnocení

Dominují opakovaná selhání ve stejném režimu (49 z 54).

Důsledek pro další vývoj

Opravné prompty vyžadují silnější ukotvení na úrovni parametrů a schémat nástrojů.

Výkonnost běhu a testovací sada

Metrika	Hodnota
Hodnocené případy	40/40
Hardware	Colab Tesla T4 (15 GB)
Celkový čas	16,8 min
Tokeny na úspěch	∼7,356
Sada testů	255 testů
Verze GRASS GIS	8.5.0

Správa a řízení testů

- Jednotkové, integrační a kontraktní testy.
- Neprivilegovaný účet `grader`.
- Příprava prostředí přes `run_remote.sh`.

Deterministické výkazy

Parametr `--report-only` vygeneruje zprávu z uložených dat bez volání LLM nebo GRASS.

Integrita výkaznictví

Generování zpráv ověřeno jako bajtově identické při opakovaném zpracování.

Co obsahuje sada 255 testů

Oblast	Co ověřuje	Počet
Klasifikace a chyby	4 režimy, precedence, lokalizace, idempotence	112
Self-healing a ukotvení	4 prvky opravy, Zero-Chat, inventář, cílený dotaz	49
Vykonávání a izolace	segfault, timeout, PERMANENT hash, scratch, offline	21
Kontrakty a perzistence	přepínače CLI, fixtures, atomický zápis, obnovení	58
Reportování	Pass@1/Healed, počty režimů, přechody, výhrada ke korektnosti	15
Celkem v repozitáři		**255**

K čemu testy slouží

Garantují správnost architektury, spolehlivou detekci pádů (C-signály, timeouty), hermetickou izolaci sandboxu a reprodukovatelnost výsledků bez nutnosti manuálního ověřování.

Kdy a jak se spouští

- **Při vývoji (lokálně):** Spouští se přes `pytest` před každou změnou architektury.
- **Před během benchmarku (Colab):** Skript `run_remote.sh` je automaticky spustí v prostředí GRASS GIS 8.5.0 pod neoprávněným účtem `grader` pro ověření integrity prostředí ještě před startem hodnocení.

Diskuze: co Gate 2 potvrzuje

1. Rozdíl oproti statické analýze je potvrzen

Statická validita 90,0 % vs. spustitelnost na 1. pokus 27,5 %. Syntaxe a schéma nezaručují spuštění v prostředí.

2. Ukotvení a self-healing mají měřitelný přínos

Ukotvení v inventáři snížilo `missing_input` z 56 na 31; ohraničený self-healing přidal +5,0 p. b. (+18,2 % relativně).

3. Bezpečnost spuštění je prokázána

Bezhlavý CLI sandbox, izolace procesních skupin, nezměněný hash `PERMANENT` a nulové úniky dočasných adresářů.

Možnosti dalšího postupu

Možnost	Přínos	Náklady a rizika
Ukotvení parametrů v RAG	Cílení na chybějící `column=`, region a monitory	Nízké náklady; riziko kvality vyhledávání
Větší modely (14B/32B)	Testování škálování nad rámec 7B	Vysoké nároky na GPU a čas
Konsolidace a dokumentace	Shromáždění výsledků z Gate 1 a Gate 2	Nízké náklady; odkládá další vývoj

Doporučené pořadí diskuse

1. Ukotvení parametrů: nejvyšší přínos proti odhalené vrstvě `general_exception`.
2. Škálování modelů: pouze pokud rozdělení chyb ospravedlňuje operativní náklady.
3. Konsolidace výstupů: ukotvení ověřeného přínosu Gate 2.

Stopa důkazů a reprodukovatelnost

Artefakt	**Obsah**
`benchmarks/execution_report.md`	Hlavní metriky, režimy selhání, přechody, náklady na tokeny
`benchmarks/leakage_report.md`	Audit kontaminace: Jaccard, n-gramy, označené případy
`grass_agent/`	Vykonávání, `inner_wrapper.py`, inventář, self-healing
`specs/`	Specifikace, plán, výzkum, datový model, kontrakty
`tests/` + `run_remote.sh`	255 testů v repozitáři a příprava prostředí v Colab

Otázky a diskuze

Děkuji za pozornost.